Lógica para Engenharia de Software

Subgrafos, Caminhos, Ciclos, Árvore Geradora Mínima com Prim e Kruskal, Isomorfismo de Grafos e problema do Caixeiro Viajante

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



Sumário

1. Subgrafos
2. Componentes Conexas
3. Grafos Isomorfos
4. Caminhos
5. Árvores Geradoras Mínimas (AGM)
6. Ciclos e PCV

Subgrafos: Definição Formal

Definição

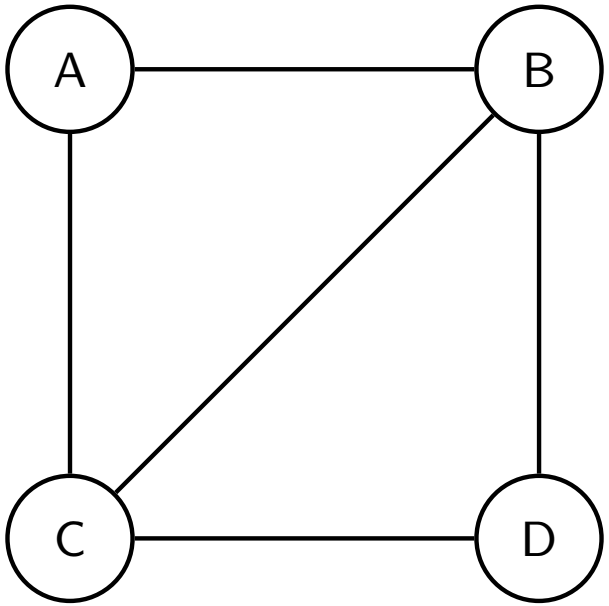
Seja $G = (V, E)$ um grafo. Um grafo $H = (V_H, E_H)$ é dito um **subgrafo** de G, denotado por $H \subseteq G$, se e somente se:

- $V_H \subseteq V$
- $E_H \subseteq E$
- $\forall e \in E_H$, se $e = \{u, v\}$, então $\{u, v\} \subseteq V_H$

A terceira condição (fecho das arestas) garante matematicamente que toda aresta em H incide exclusivamente sobre vértices que também estão presentes em H, mantendo a estrutura de grafo bem definida.

Exemplo Prático: Grafo Original G

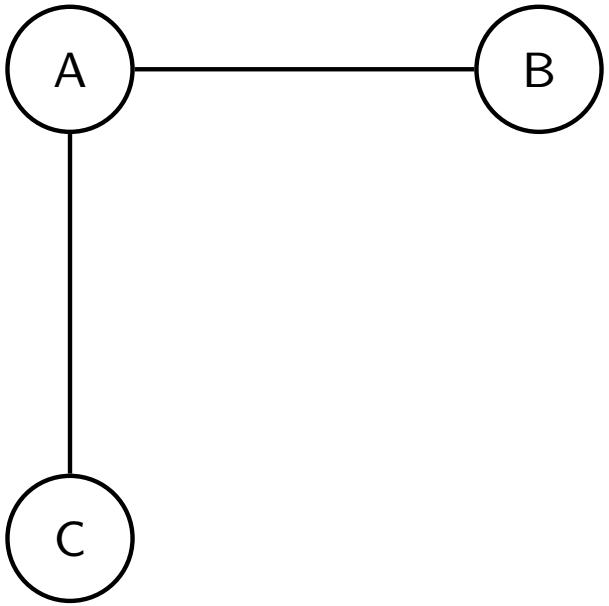
Considere o seguinte grafo $G = (V, E)$:



$V = \{A, B, C, D\}$
$E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, D\}\}$

Exemplo Prático: Subgrafo H

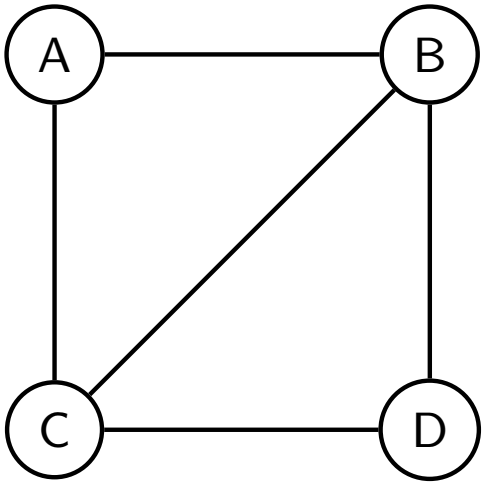
A partir de G, podemos formar o subgrafo $H = (V_H, E_H)$:



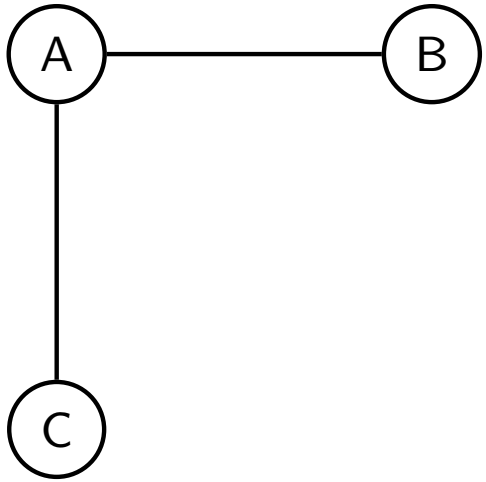
$V_H = \{A, B, C\} \subseteq \{A, B, C, D\}$
$E_H = \{\{A, B\}, \{A, C\}\} \subseteq E$

Comparação: Grafo G e Subgrafo H

Grafo G



Subgrafo H



Note que removemos o vértice **D** e as arestas $\{B, C\}$, $\{B, D\}$ e $\{C, D\}$.

Tipos Especiais de Subgrafos

Subgrafo Próprio

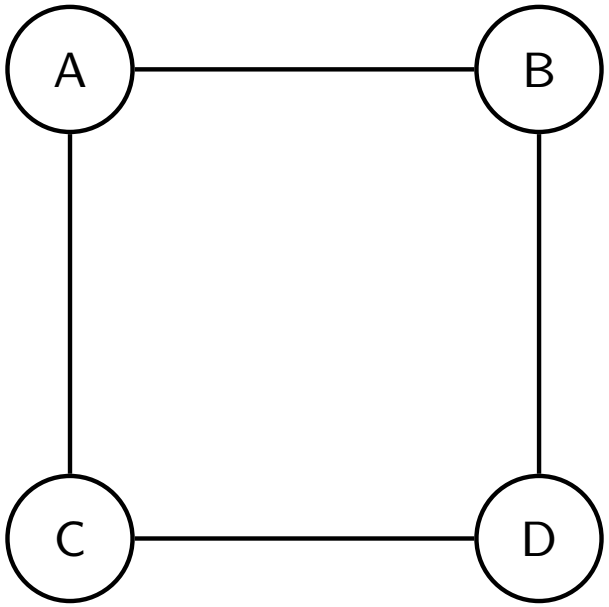
Um subgrafo H de G é chamado de **subgrafo próprio** se $H \neq G$. Ou seja, H tem menos vértices ou menos arestas que G.

Subgrafo Gerador (Spanning Subgraph)

Um subgrafo H é um **subgrafo gerador** de G se ele contém **todos os vértices** de G ($V_H = V$), mas pode ter menos arestas ($E_H \subseteq E$).

Exemplo: Subgrafo Gerador

Subgrafo H' onde $V_{H'} = \{A, B, C, D\}$ e $E_{H'} = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}$.



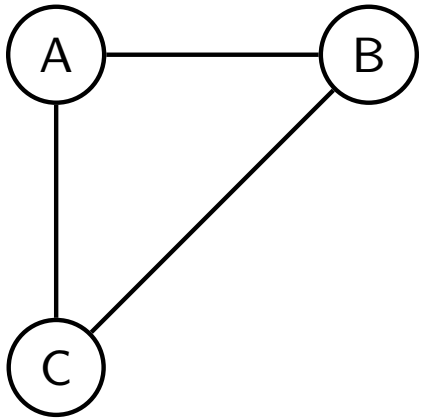
Aqui, removemos apenas a aresta $\{B, C\}$. Como todos os vértices originais estão presentes, este é um **subgrafo gerador**.

Subgrafo Induzido

Definição

Um subgrafo H de G é dito **induzido** por um conjunto de vértices $V_H \subseteq V$ se H contém **todas** as arestas de G que ligam vértices de V_H.

Exemplo: Subgrafo induzido pelo conjunto $V_H = \{A, B, C\}$.
Em G, as arestas entre $\{A, B, C\}$ são $\{A, B\}, \{A, C\}$ e $\{B, C\}$.



(O subgrafo H dos primeiros slides **não** era induzido, pois faltava a aresta $\{B, C\}$).

Resumo e Aplicações

- **Subgrafo**: Qualquer subconjunto válido de vértices e arestas.
- **Subgrafo Gerador**: Mantém todos os vértices, podendo omitir arestas (muito usado em árvores geradoras, roteamento).
- **Subgrafo Induzido**: Escolhe-se os vértices e obrigatoriamente mantém-se todas as arestas originais entre eles (útil para encontrar cliques, comunidades em redes).

Compreender subgrafos é o primeiro passo para analisar estruturas menores dentro de grandes redes complexas.

Componentes Conexas: Definição

Definição

Uma **componente conexa** de um grafo é um conjunto **maximal** de vértices que estão conectados por caminhos.

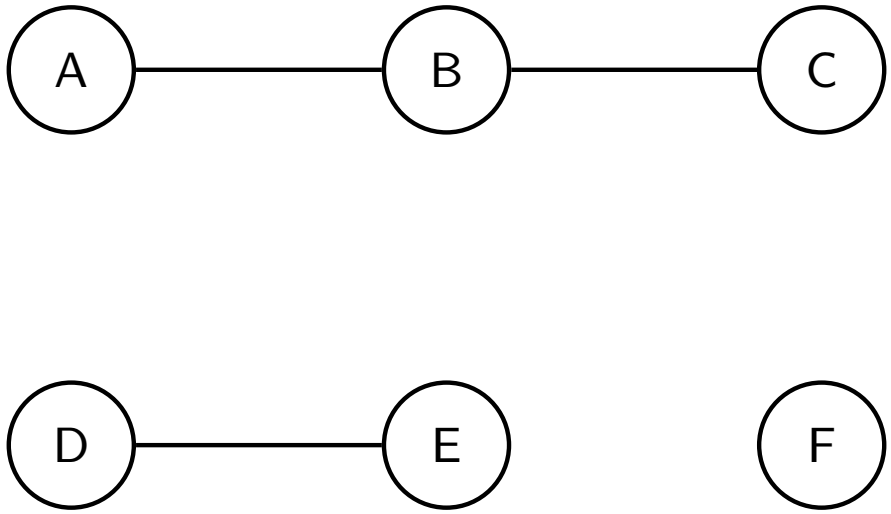
O que significa "maximal"?

Significa que não é possível adicionar mais nenhum outro vértice do grafo a esse conjunto sem perder a propriedade de que todos estão conectados entre si por algum caminho.

- Se o grafo todo é conexo, ele possui apenas **1** componente conexa.
- Se o grafo é desconexo, ele possui **2 ou mais** componentes conexas.

Exemplo Visual: Grafo Desconexo

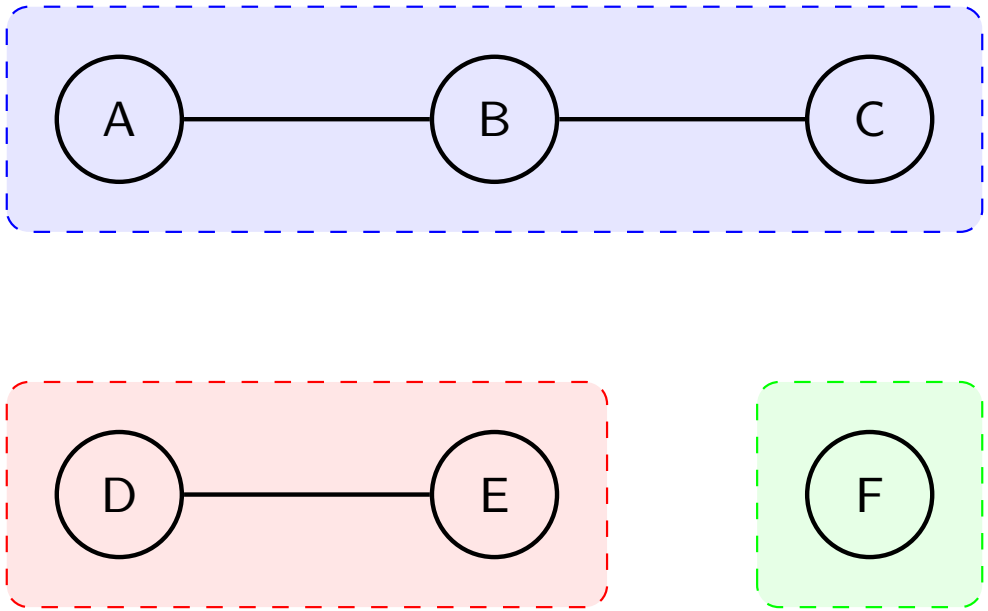
Considere o seguinte grafo G:



Embora pareçam três figuras separadas, matematicamente isso compõe **um único grafo** G.

Identificando as Componentes Conexas

Neste grafo, podemos identificar claramente três grupos maximais isolados:



Componentes: $C_1 = \{A, B, C\}$, $C_2 = \{D, E\}$, $C_3 = \{F\}$.

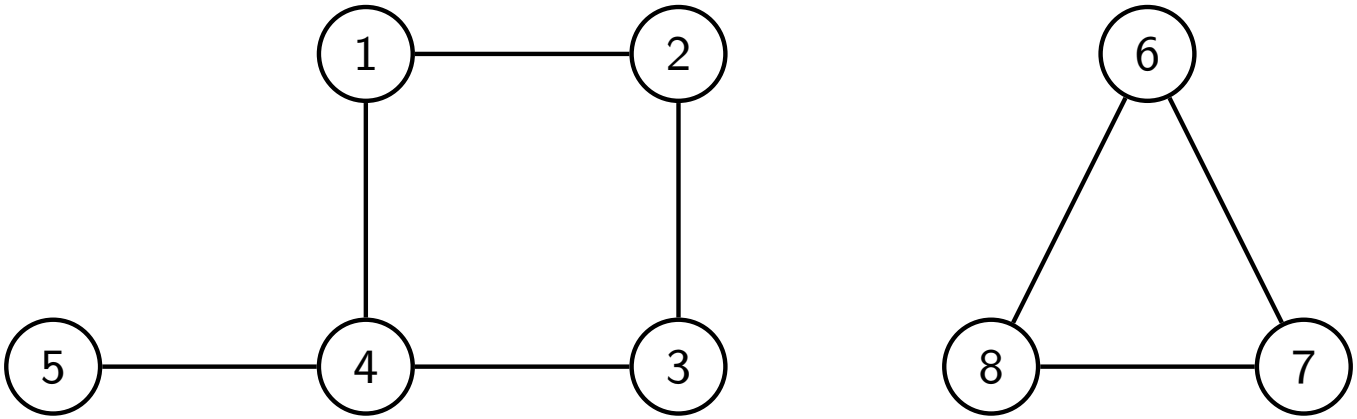
Aplicações Práticas

O estudo de componentes conexas é fundamental na Engenharia de Software e em outras áreas para entender o isolamento e a comunicação em sistemas:

- **Redes Sociais**: Identificar bolhas ou comunidades isoladas (grupos de pessoas onde ninguém conhece ninguém fora do grupo).
- **Redes Elétricas**: Detectar partes da malha elétrica que ficaram sem energia ou isoladas após a falha de uma conexão principal.
- **Sistemas de Transporte**: Verificar se todas as cidades de uma malha viária ou aeroportuária são acessíveis a partir de qualquer outra.

Exercício 1: Componentes Conexas

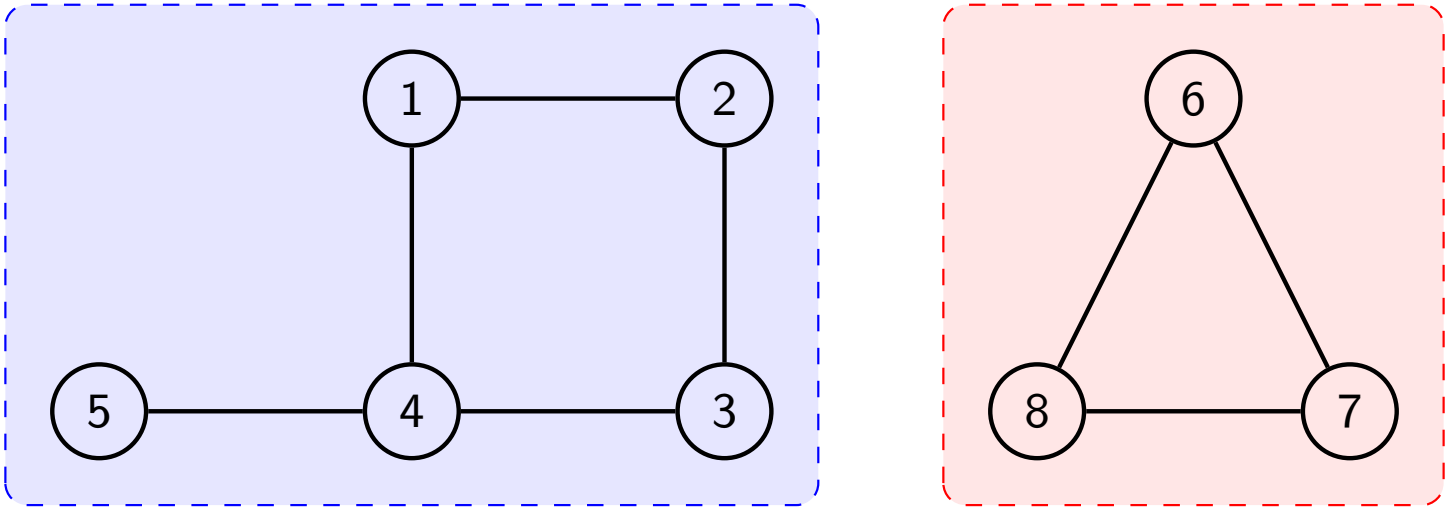
Dado o grafo G abaixo:



Pergunta: Identifique as componentes conexas deste grafo. Escreva o conjunto de vértices pertencente a cada componente.

Resolução 1: Componentes Conexas

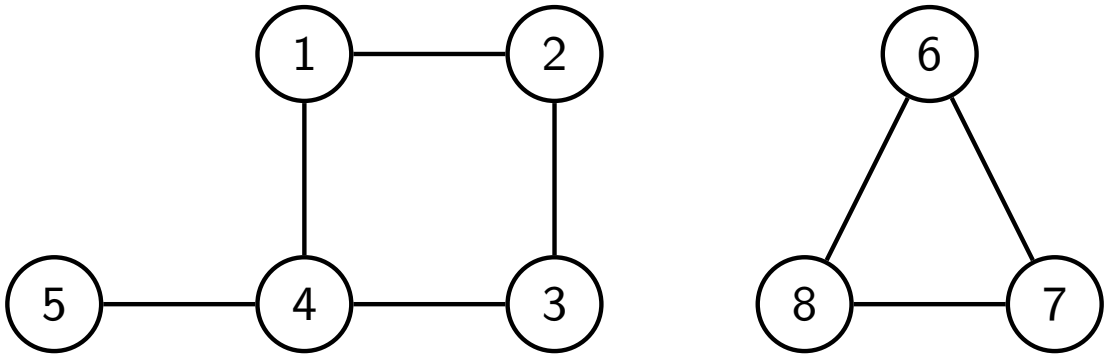
O grafo possui **duas** componentes conexas distintas:



Resposta: $C_1 = \{1, 2, 3, 4, 5\}$ e $C_2 = \{6, 7, 8\}$.

Exercício 2: Construção de Subgrafos

Utilizando o **mesmo grafo** G da questão anterior:

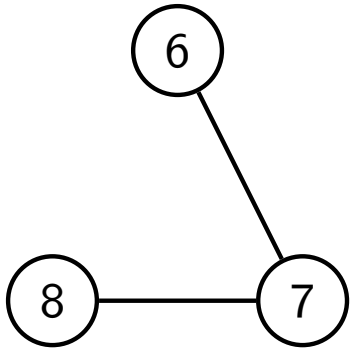


Construa três subgrafos diferentes com as seguintes regras:

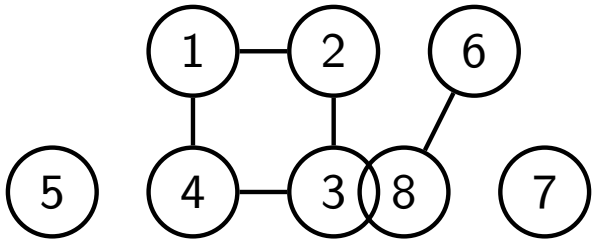
1. H_1: Um subgrafo qualquer contendo apenas 3 vértices.
2. H_2: Um subgrafo gerador do grafo G.
3. H_3: O subgrafo induzido pelo conjunto de vértices $V_{H_3} = \{1, 4, 5\}$.

Resolução 2: Construção de Subgrafos

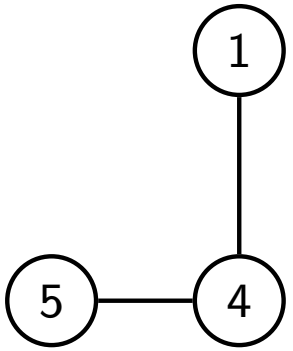
1) H_1 (3 vértices)



2) H_2 (Gerador)



3) H_3 (Induzido)



Nota: As respostas para H_1 e H_2 são apenas exemplos, pois existem várias possibilidades corretas. H_3 possui uma única resposta.

Grafos Isomorfos

Objetivos

- Compreender o conceito de **isomorfismo** em teoria dos grafos.
- Aplicar técnicas e estratégias de verificação para determinar se dois grafos são isomorfos.

A Intuição do Isomorfismo

- Muitas vezes, dois grafos desenhados de maneiras completamente diferentes na verdade representam a **mesma estrutura**.
- Se pudermos "mover"os vértices de um grafo (esticando ou dobrando as arestas, mas sem cortá-las) de modo que ele fique exatamente igual ao outro, os dois grafos são **isomorfos**.
- Isomorfismo vem do grego: *isos* (igual) e *morphos* (forma).

Definição

Definição Formal

Dois grafos G e H são **isomorfos** quando possuem exatamente a **mesma estrutura** topológica, mesmo que seus vértices tenham nomes diferentes ou sejam desenhados de formas diferentes.

Em outras palavras, se os grafos forem "despidos"dos rótulos de seus vértices, não haverá como distinguir um do outro.

A Função Bijetora

Matematicamente, G e H são isomorfos se existir uma **função bijetora**:

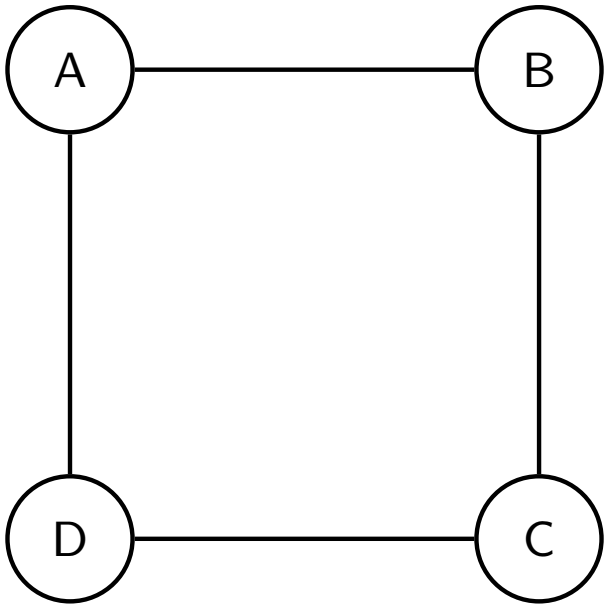
$$f : V(G) \to V(H)$$

Que **preserva as adjacências**. Isso significa que:

- Dois vértices u, v são vizinhos em G **se, e somente se**, $f(u)$ e $f(v)$ são vizinhos em H.
- A função pareia cada vértice de G com exatamente um vértice de H e vice-versa.

Exemplo: Grafo G

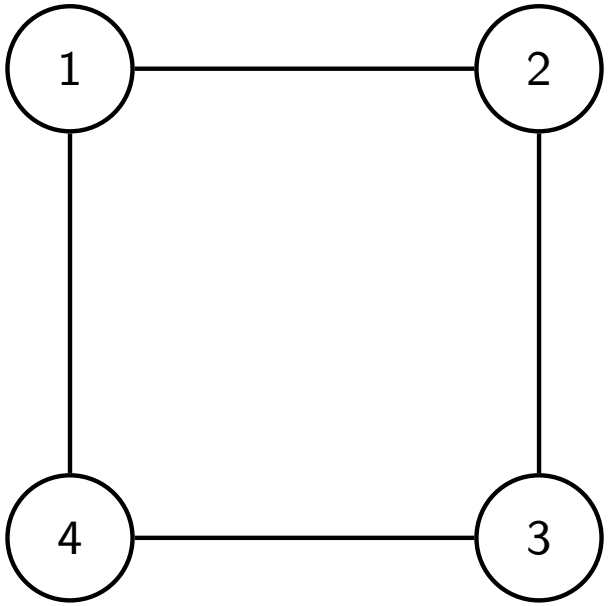
Considere o grafo G:



Este é um ciclo com 4 vértices (um quadrado).

Exemplo: Grafo H

Agora considere o grafo H:



À primeira vista, os rótulos são diferentes, mas a estrutura de conexões é idêntica ao grafo G.

Mapeamento de Adjacências ($G \to H$)

Para provar o isomorfismo, aplicamos o seguinte mapeamento (a função bijetora f):

Grafo G	$\to$	**Grafo** H
A	$\to$	1
B	$\to$	2
C	$\to$	3
D	$\to$	4

Verifique as adjacências: A e B estão conectados, assim como 1 e 2 estão. C e D estão conectados, assim como 3 e 4 estão. Todas as conexões são perfeitamente preservadas.

Estratégia de Verificação

Antes de tentar encontrar o mapeamento, podemos verificar rapidamente algumas **condições necessárias**.
Dois grafos **só podem** ser isomorfos se tiverem:

- O mesmo **número de vértices** ($|V(G)| = |V(H)|$).
- O mesmo **número de arestas** ($|E(G)| = |E(H)|$).
- A mesma **sequência de graus** (os graus dos vértices ordenados devem ser idênticos).
- A mesma **estrutura de ciclos** (ex: mesmo número de triângulos, quadrados).

*Nota: Se alguma condição falhar, os grafos **não** são isomorfos.*

Algoritmo Combinatorial Simples

Caso as condições iniciais sejam atendidas, o algoritmo básico (força bruta) para provar o isomorfismo é:

1. Compare a quantidade de vértices.
2. Compare a quantidade de arestas.
3. Compare a sequência de graus.
4. **Teste permutações:** Gere todas as formas possíveis de parear os vértices de G com os vértices de H.
5. **Verifique adjacências:** Para cada permutação, teste se as adjacências são preservadas. O primeiro mapeamento que funcionar confirma o isomorfismo.

Complexidade do Algoritmo Combinatorial

O passo de testar permutações é o grande gargalo da verificação de isomorfismo em computação:

Complexidade

A complexidade deste algoritmo força bruta é $\mathcal{O}(n!)$ (onde n é o número de vértices).

- Para um grafo com 10 vértices: $10! = 3.628.800$ permutações.
- Para um grafo com 20 vértices: $\approx 2.4 \times 10^{18}$ permutações (impraticável!).

É por isso que o *Problema do Isomorfismo de Grafos* é famoso na Ciência da Computação.

Caminhos, Ciclos e Ciclos Hamiltonianos

Objetivos da Aula

- Diferenciar os conceitos de **caminhos** e **ciclos** em grafos.
- Identificar e compreender o que torna um ciclo **Hamiltoniano**.
- Compreender como esses conceitos são aplicados para resolver problemas clássicos da Computação.

O que é um Caminho?

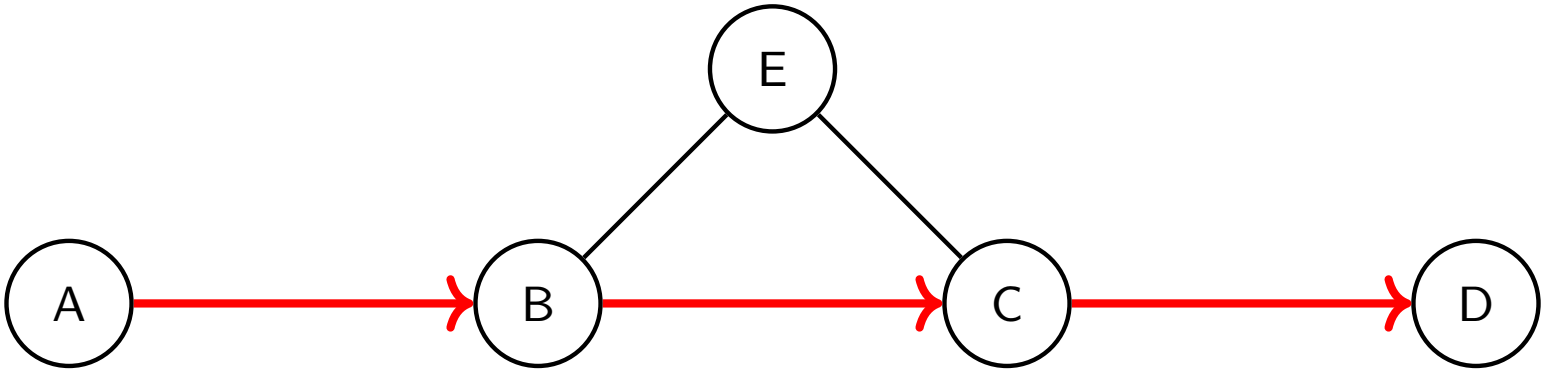
Definição de Caminho

Um **caminho** em um grafo é uma sequência de vértices adjacentes conectados por arestas, onde nos movemos de um vértice para outro seguindo a estrutura do grafo.

- É literalmente a "rota"que pode ser percorrida partindo de um nó origem até um nó destino.
- Formalmente, é uma sequência $v_1, v_2, ..., v_k$ onde cada par adjacente (v_i, v_{i+1}) é uma aresta do grafo.

Exemplo Visual: Caminho

Exemplo: O caminho $A \to B \to C \to D$



As setas em **vermelho** destacam a sequência de vértices visitados.

Comprimento de um Caminho

Como medimos um caminho?

O **comprimento** de um caminho é determinado pelo **número de arestas** que o compõem, e não pelo número de vértices.

Voltando ao exemplo $A \to B \to C \to D$:

- Vértices visitados: 4 (A, B, C, D).
- Arestas percorridas: 3 $((A, B), (B, C), (C, D))$.
- **Comprimento = 3**.

Árvore Geradora Mínima

Objetivos

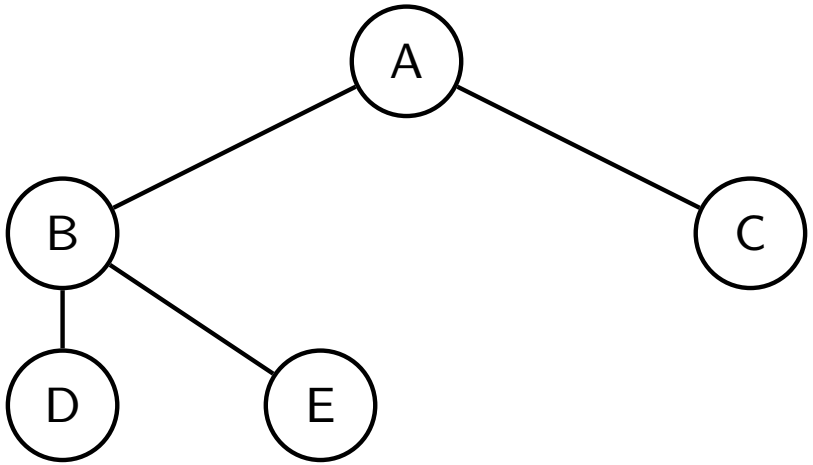
- Compreender o conceito de **árvore geradora** e **árvore geradora mínima** (AGM).
- Entender intuitivamente os algoritmos de **Kruskal** e **Prim**.
- Reconhecer as aplicações práticas da AGM em problemas de engenharia.

O que é uma Árvore? (Revisão)

Definição de Árvore

Uma **árvore** é um grafo **conexo** e **acíclico**. Ou seja, é um grafo que:

- Possui **exatamente** um caminho entre cada par de vértices.
- **Não** contém ciclos.
- Se possui n vértices, possui exatamente $n - 1$ arestas.



Árvore Geradora (Spanning Tree)

Definição

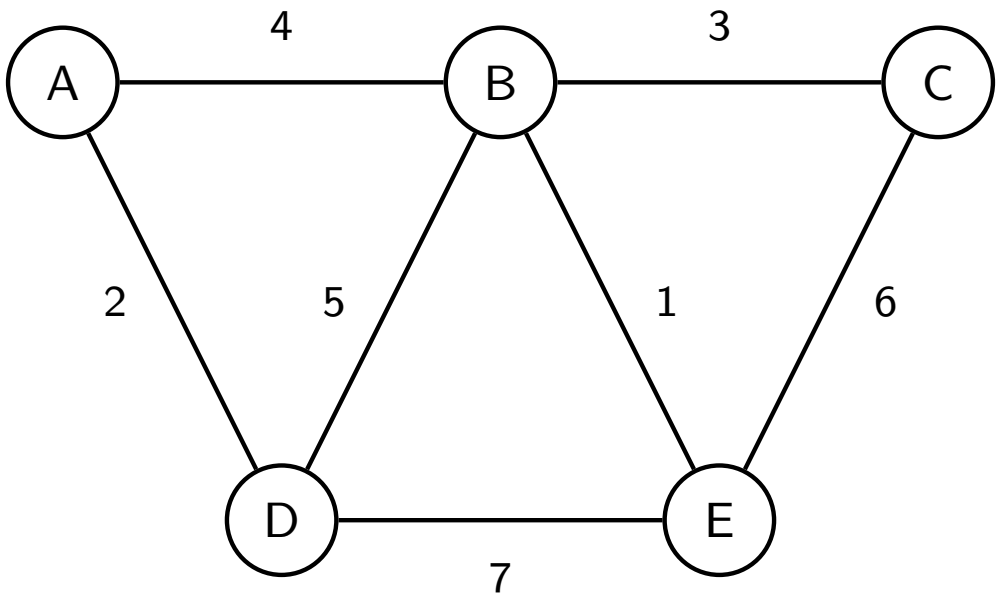
Dado um grafo conexo $G = (V, E)$, uma **árvore geradora** T de G é um **subgrafo gerador** de G que é ao mesmo tempo uma **árvore**. Ou seja:

- T contém **todos os vértices** de G ($V_T = V$).
- T é conexo e **não possui ciclos**.
- T possui exatamente $|V| - 1$ arestas.

Um grafo pode ter **múltiplas** árvores geradoras diferentes.

Exemplo: Grafo com Pesos

Considere o grafo G abaixo com pesos nas arestas:



O peso de uma aresta representa um custo (distância, latência, etc.).

Árvore Geradora Mínima (AGM)

Definição Formal

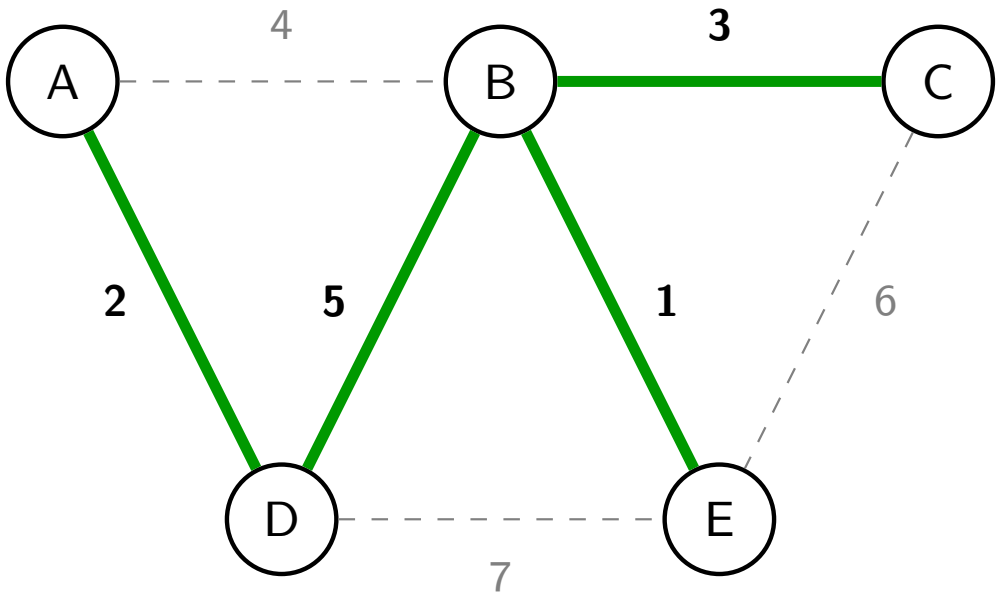
Dado um grafo conexo e ponderado $G = (V, E, w)$, onde $w : E \to \mathbb{R}$ atribui pesos às arestas, a **Árvore Geradora Mínima** (AGM) é uma árvore geradora T^* cujo **peso total** é mínimo:

$$T^* = \arg \min_{T \subseteq G,\ T \text{ é A.G.}} \sum_{e \in T} w(e)$$

- A AGM **conecta todos os vértices** com o menor custo total possível.
- Em inglês: *Minimum Spanning Tree* (MST).

AGM do Exemplo Anterior

A AGM do grafo anterior utiliza as arestas de **menor custo** que ainda conectam todos os vértices:



Arestas selecionadas: $\{A, D\} = 2$, $\{B, E\} = 1$, $\{B, C\} = 3$, $\{B, D\} = 5$. **Custo total = 11**.

Algoritmo de Kruskal

```
KRUSKAL(G, w)
1    A ← ∅
2    para cada vértice v ∈ G.V
3       CRIAR-CONJUNTO(v)
4    ordene as arestas de G.E em ordem não decrescente de peso w
5    para cada aresta (u, v) ∈ G.E, na ordem não decrescente de w
6       se ENCONTRAR-CONJUNTO(u) ≠ ENCONTRAR-CONJUNTO(v)
7          A ← A ∪ {(u, v)}     // não forma ciclo:  adiciona
8          UNIR(u, v)     // une as duas componentes
9    retorne A
```

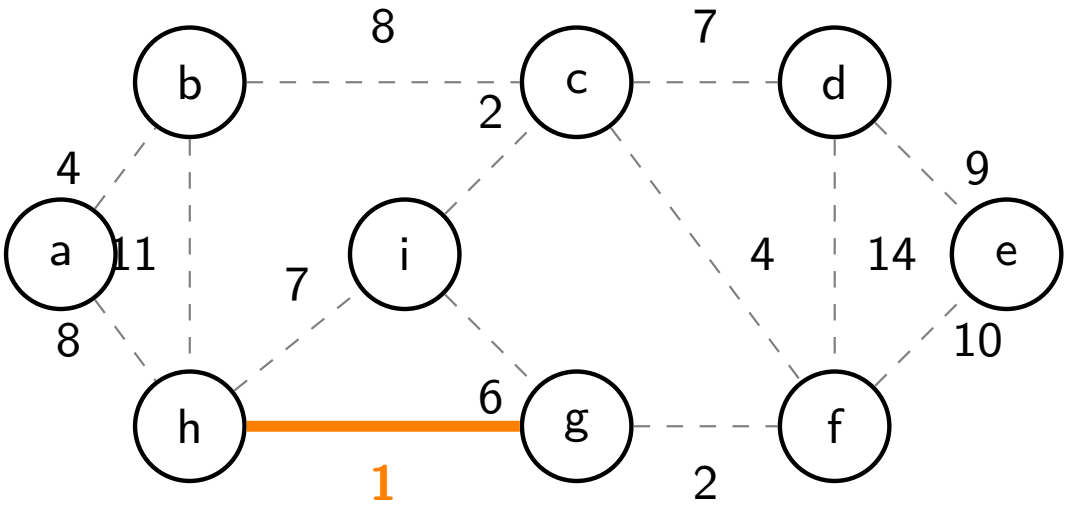
Estruturas auxiliares

- CRIAR-CONJUNTO(v): inicializa cada vértice como sua própria componente.
- ENCONTRAR-CONJUNTO(u): retorna o representante da componente de u.
- UNIR(u, v): funde as componentes de u e v (*Union-Find*).

Kruskal: Passo 1/8 — Aresta $\{h, g\} = 1$

⟨ **(h,g,1)**, (c,i,2), (g,f,2), (a,b,4), (c,f,4), (i,g,6), (c,d,7), (h,i,7), (a,h,8), (b,c,8), (d,e,9) ⟩

Grafo



Conjuntos

$\{a\}$ $\{b\}$ $\{c\}$ $\{d\}$ $\{e\}$ $\{f\}$ $\{i\}$

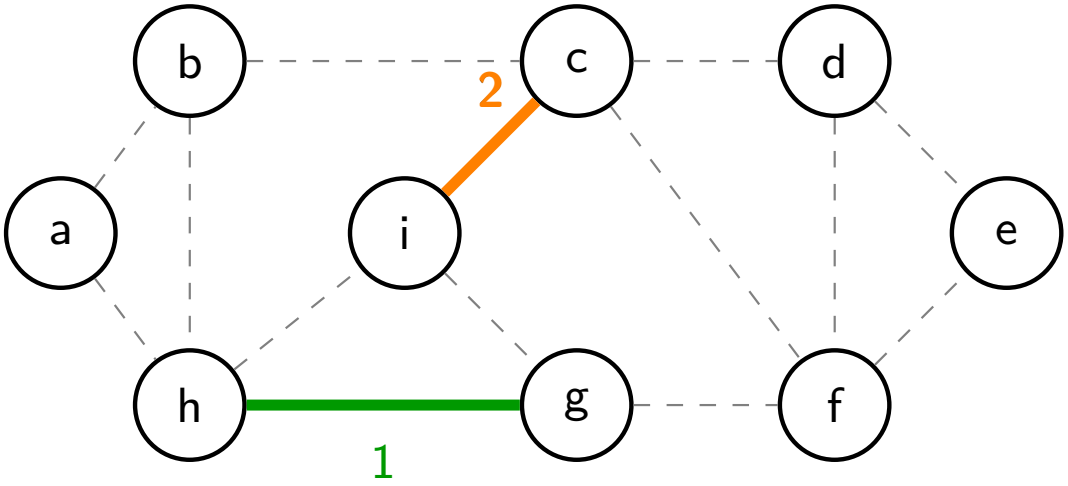
$\boxed{\{h, g\}}$

Decisão

$\{h, g\}$ estão em conjuntos distintos.
Adicionar aresta.
Custo $= 1$

Kruskal: Passo 2/8 — Aresta $\{c, i\} = 2$

⟨ ✓(h,g,1), **(c,i,2)**, (g,f,2), (a,b,4), (c,f,4), (i,g,6), (c,d,7) ⟩



$\boxed{\{h, g\}}$

$\boxed{\{c, i\}}$

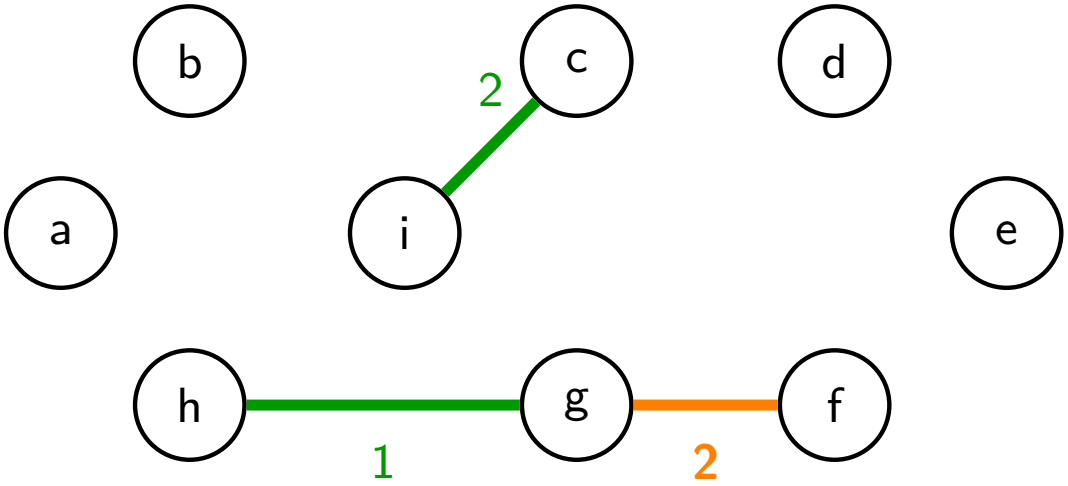
$\{a\}\{b\}\{d\}\{e\}\{f\}$

Decisão

Adicionar $\{c, i\}$.
Custo acumulado $= 3$

Kruskal: Passo 3/8 — Aresta $\{g, f\} = 2$

⟨ ✓(h,g,1), ✓(c,i,2), **(g,f,2)** ⟩



$\boxed{\{h, g, f\}}$

$\boxed{\{c, i\}}$

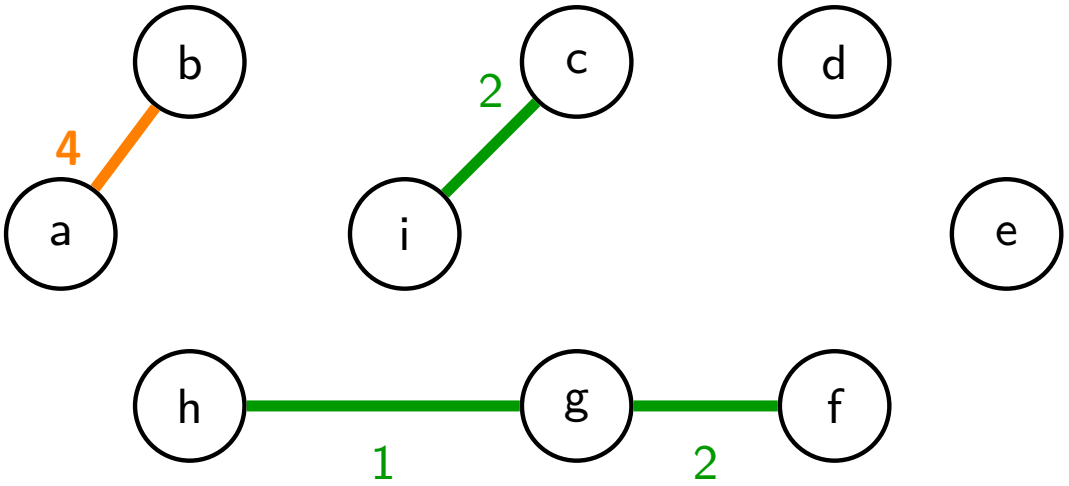
$\{a\}\{b\}\{d\}\{e\}$

Decisão

Adicionar $\{g, f\}$.
Custo acumulado $= 5$

Kruskal: Passo 4/8 — Aresta $\{a, b\} = 4$

⟨ ✓(h,g,1), ✓(c,i,2), ✓(g,f,2), **(a,b,4)** ⟩



$\{h, g, f\}$

$\{c, i\}$

$\{a, b\}$

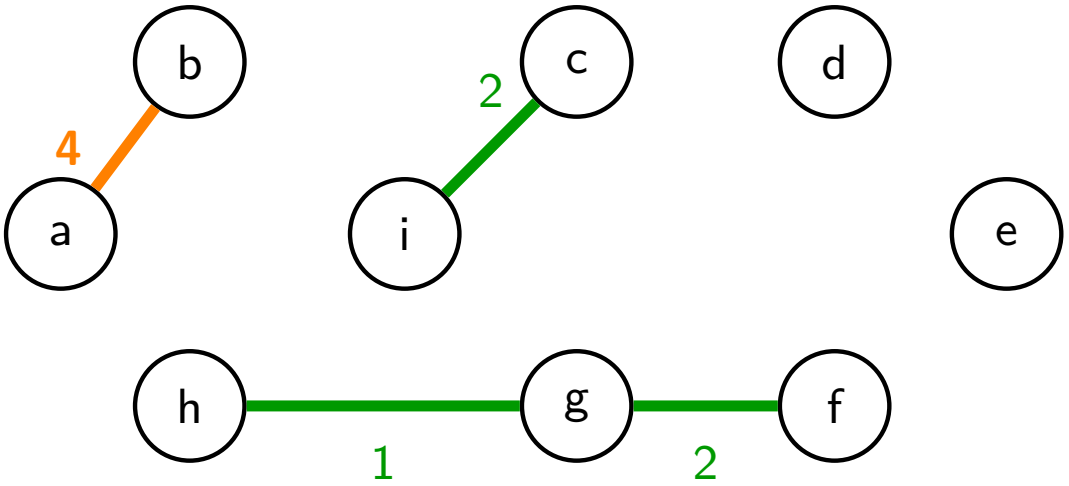
$\{d\}\{e\}$

Decisão

Adicionar $\{a, b\}$.
Custo acumulado $= 9$

Kruskal: Passo 4/8 — Aresta $\{a, b\} = 4$

⟨ ✓(h,g,1), ✓(c,i,2), ✓(g,f,2), **(a,b,4)** ⟩



$\{h, g, f\}$

$\{c, i\}$

$\{a, b\}$

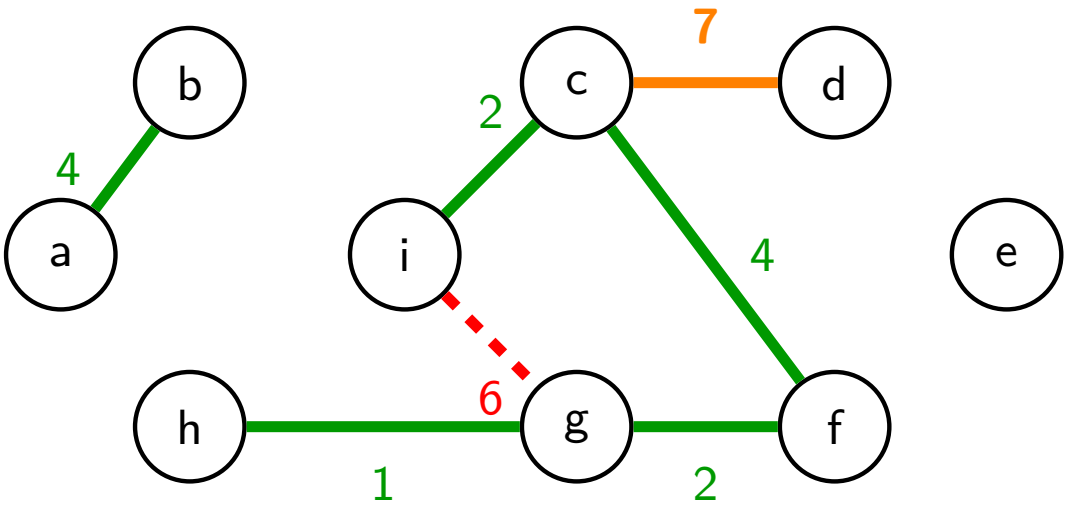
$\{d\}\{e\}$

Decisão

Adicionar $\{a, b\}$.
Custo acumulado $= 9$

Kruskal: Passo 6/8 — Aresta $\{c, d\} = 7$

⟨ ✓(h,g,1), ✓(c,i,2), ✓(g,f,2), ✓(a,b,4), ✓(c,f,4), ~~(i,g,6)~~, **(c,d,7)** ⟩



$\{c, d, f, g, h, i\}$

$\{a, b\}$

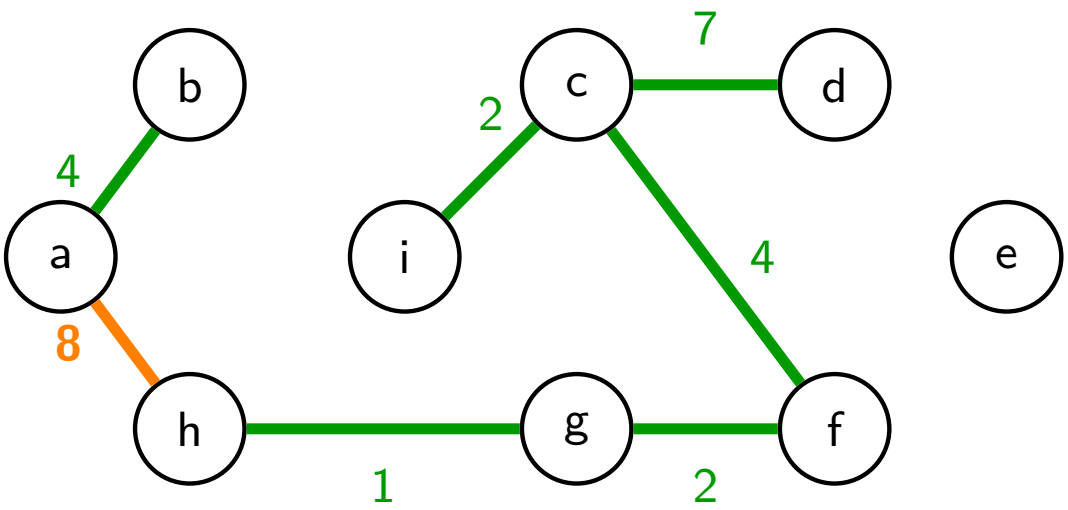
$\{e\}$

Decisão

$\{i, g\}$ gera ciclo.
Descartar.
Adicionar $\{c, d\}$.
Custo acumulado $= 20$

Kruskal: Passo 7/8 — Aresta $\{a, h\} = 8$

⟨ ✓(h,g,1), ✓(c,i,2), ✓(g,f,2), ✓(a,b,4), ✓(c,f,4), ~~(i,g,6)~~, ✓(c,d,7), ~~(h,i,7)~~, **(a,h,8)** ⟩



$\{a, b, c, d, f, g, h, i\}$

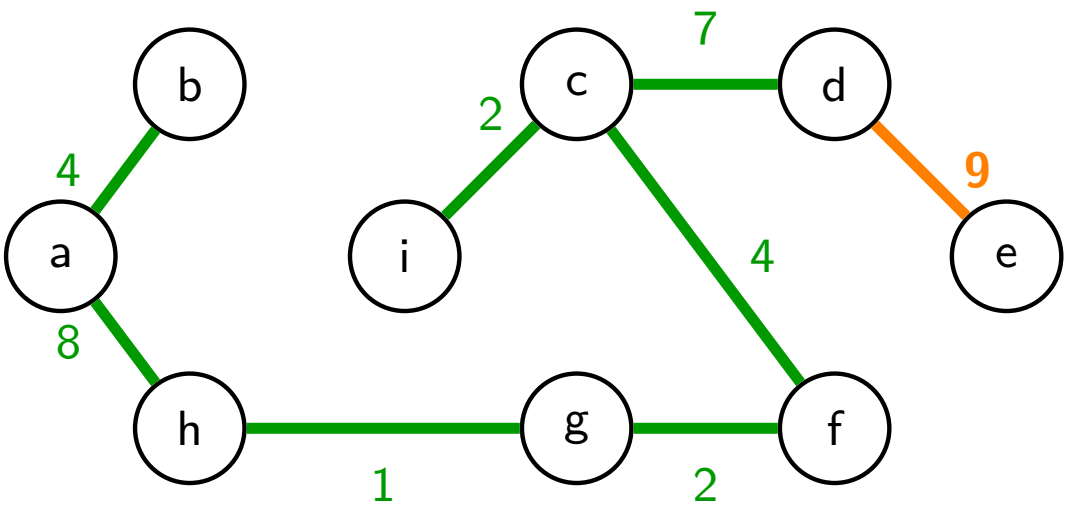
$\{e\}$

Decisão

$\{h, i\}$ gera ciclo.
Descartar.
Adicionar $\{a, h\}$.
Custo acumulado $= 28$

Kruskal: Passo 8/8 — Aresta $\{d, e\} = 9$ (AGM Completa)

⟨ ✓(h,g,1), ✓(c,i,2), ✓(g,f,2), ✓(a,b,4), ✓(c,f,4), ~~(i,g,6)~~, ✓(c,d,7), ~~(h,i,7)~~, ✓(a,h,8), ~~(b,c,8)~~, **(d,e,9)** ⟩



$\{a, b, c, d, e, f, g, h, i\}$

Resultado

Adicionar $\{d, e\}$.
Todos os vértices estão conectados.

Número de arestas:

$$|V| - 1 = 8$$

Custo total:

$1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = \mathbf{37}$

Algoritmo de Prim

Estratégia: Greedy por Vértices

O algoritmo de **Prim** constrói a AGM expandindo **um vértice por vez**. A cada passo, escolhe a aresta de menor peso que conecta a árvore atual a um vértice ainda **não visitado**.

Passos do algoritmo:

1. Escolha qualquer vértice inicial. Marque-o como visitado.
2. Considere todas as arestas que partem dos vértices visitados.
3. Selecione a aresta de **menor peso** que chega a um vértice **não visitado**.
4. Marque o novo vértice como visitado e repita até incluir todos.

Comparativo: Kruskal × Prim

Característica	**Kruskal**	**Prim**
Estratégia	Por arestas	Por vértices
Estrutura auxiliar	Union-Find	Fila de prioridade
Complexidade	$\mathcal{O}(E \log E)$	$\mathcal{O}(E \log V)$
Ideal para	Grafos esparsos	Grafos densos
Resultado	Mesmo (AGM)	Mesmo (AGM)

- Ambos os algoritmos são **gulosos** (*greedy*) e produzem a mesma AGM.
- A prova de corretude baseia-se na **Propriedade do Corte** e na **Propriedade do Ciclo** da teoria dos grafos.

Aplicações da AGM

A Árvore Geradora Mínima aparece em **problemas reais** de engenharia e computação:

- **Redes de telecomunicações**: Conectar n cidades com o menor comprimento total de cabo de fibra óptica.
- **Redes elétricas**: Projeto de redes de distribuição de energia com custo mínimo de instalação.
- **Cluster em Machine Learning**: O algoritmo *single-linkage clustering* utiliza a AGM para agrupar dados similares.
- **Roteamento de redes**: Protocolos como o **OSPF** usam conceitos de árvore geradora para evitar loops e otimizar o tráfego.
- **Circuitos impressos (PCB)**: Conectar pinos de um circuito com o menor comprimento total de trilhas.

O que é um Ciclo?

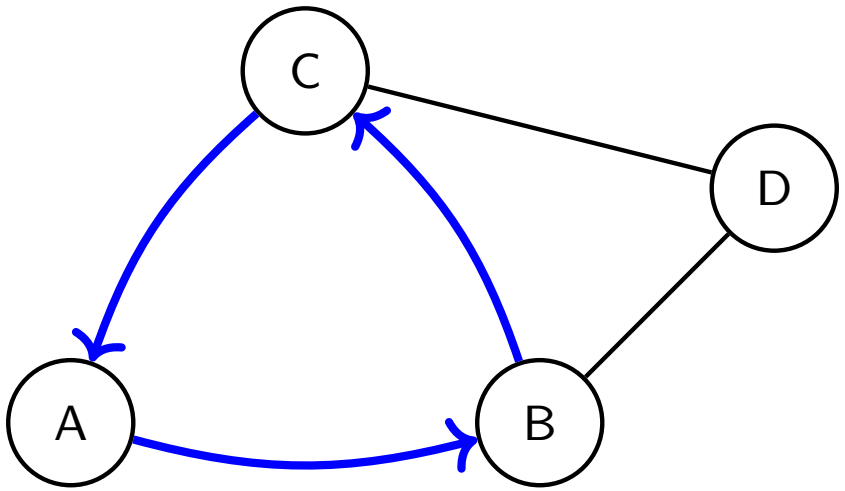
Definição de Ciclo

Um **ciclo** é simplesmente um **caminho fechado**. Ou seja, é um caminho que começa e termina no mesmo vértice.

- O tamanho (comprimento) do ciclo também é contado pelo número de arestas.
- Pelo menos 3 vértices são necessários para formar um ciclo em grafos simples não-direcionados.

Exemplo Visual: Ciclo

Exemplo: O ciclo $A \to B \to C \to A$



As setas em **azul** mostram o caminho fechado (Comprimento = 3).

Ciclos Simples

Definição de Ciclo Simples

Um ciclo é dito **simples** se ele **não repete vértices** e não repete arestas ao longo de seu percurso, exceto obviamente pelo vértice de início e fim.

- $A \to B \to C \to A$ (Simples).
- $A \to B \to C \to B \to D \to A$ (**Não** é simples, pois o vértice B foi visitado duas vezes).

Ciclo Hamiltoniano

Definição de Ciclo Hamiltoniano

Um **ciclo Hamiltoniano** é um ciclo especial e altamente restritivo: ele é um ciclo que passa por **TODOS** os vértices do grafo **exatamente uma vez** e retorna ao vértice de origem.

- Leva o nome do matemático Sir William Rowan Hamilton.
- Diferente do *Ciclo Euleriano* (que precisa passar por todas as **arestas**), o Hamiltoniano foca em visitar todos os **vértices**.

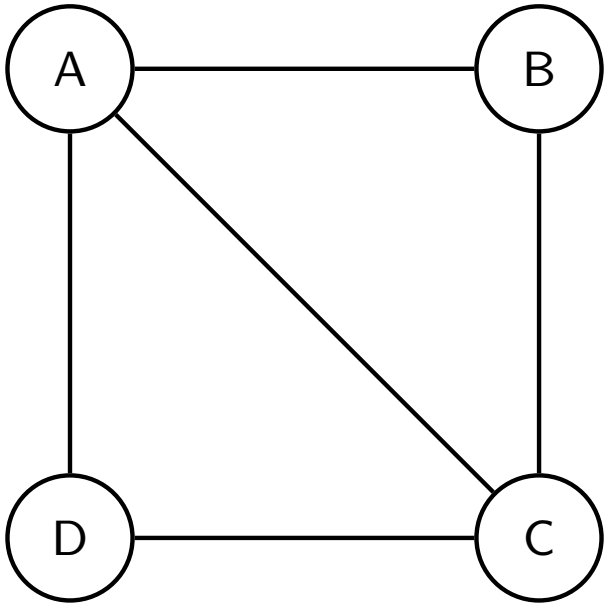
As Regras do Ciclo Hamiltoniano

Para ser considerado um Ciclo Hamiltoniano, as seguintes regras são absolutas:

1. Tem que ser um ciclo (começa e termina no mesmo lugar).
2. Tem que visitar 100% dos vértices disponíveis no grafo.
3. **Nenhum** vértice pode ser visitado mais de uma vez (com exceção da repetição obrigatória do vértice de origem no final).

Exemplo: Grafo para Busca Hamiltoniana

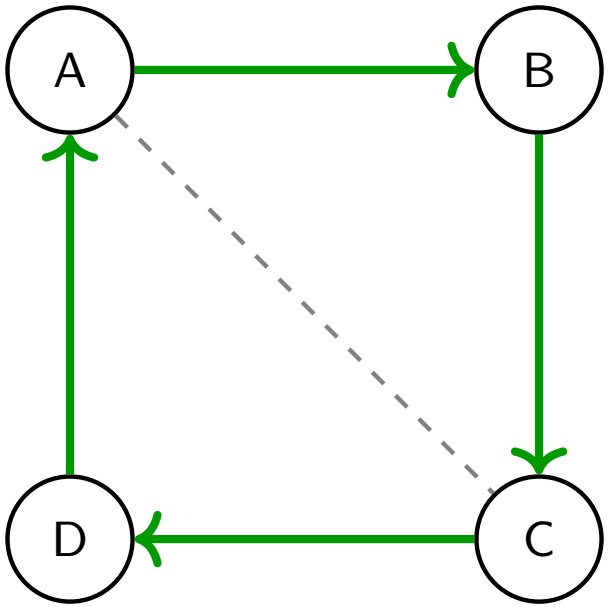
Dado o grafo abaixo, existe um ciclo Hamiltoniano?



Precisamos encontrar uma rota que passe por A, B, C e D apenas uma vez e volte ao início.

O Ciclo Hamiltoniano Destacado

Sim, existe! A sequência $A \to B \to C \to D \to A$ forma um ciclo Hamiltoniano.



Note que a aresta $\{A, C\}$ **não** foi utilizada. Em um ciclo Hamiltoniano, não precisamos percorrer todas as arestas, apenas todos os vértices.

O Problema do Caixeiro Viajante (PCV)

O conceito de ciclo Hamiltoniano é a base teórica de um dos problemas mais famosos da computação: **O Problema do Caixeiro Viajante (PCV)**.

O Problema

"Dada uma lista de cidades e a distância entre cada par delas, qual é a rota mais curta possível que visita cada cidade **exatamente uma vez** e retorna à cidade de origem?"

Relacionamento PCV e Ciclo Hamiltoniano

Como esses dois conceitos se ligam?

- O Caixeiro Viajante está buscando um **Ciclo Hamiltoniano**.
- No PCV, o grafo possui pesos nas arestas (distâncias, custos, pedágios).
- O objetivo não é encontrar *qualquer* ciclo Hamiltoniano (pois em um grafo completo existem muitos), mas sim encontrar o ciclo Hamiltoniano com a **menor soma de pesos** (a rota mais barata/curta).

Por que o PCV e a busca por Ciclos Hamiltonianos são tão estudados?

Classe NP-Completo

A determinação da existência de um ciclo Hamiltoniano em um grafo qualquer é um problema **NP-Completo**.

Isso significa que, assim como o problema do isomorfismo que vimos antes, **não existe** um algoritmo rápido (em tempo polinomial) conhecido para resolvê-lo garantidamente em grafos grandes.

Resumo: O Desafio do Tempo de Execução

- À medida que o número de vértices (cidades) cresce, o tempo necessário para testar todas as combinações no PCV explode combinatoriamente (Complexidade fatorial $\mathcal{O}(n!)$ na busca por força bruta).

- Para resolver problemas NP-Completos na vida real (como roteamento de caminhões dos Correios, design de microchips ou logística da Uber), usamos **heurísticas** e **algoritmos de aproximação** (ex: Algoritmos Genéticos), que encontram rotas "boas o suficiente"em tempo viável, mas não garantem a perfeição matemática.

Algoritmo: Busca de Ciclo Hamiltoniano

Abordagem por *Backtracking* (Tentativa e Erro):

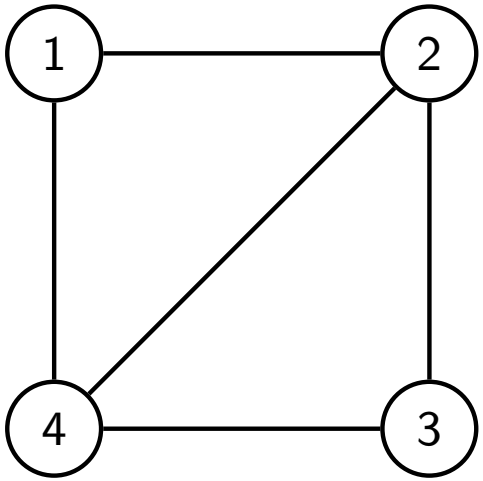
```
Função EncontrarCiclo(caminho, vertice_atual):
    Se caminho.tamanho == Total_de_Vertices:
      Se vertice_atual tem aresta para caminho[0]:
        Adicionar caminho[0] ao caminho
        Retornar Verdadeiro  // Ciclo encontrado!
      Senão:
        Retornar Falso

    Para cada vizinho de vertice_atual:
      Se vizinho não está no caminho:
        Adicionar vizinho ao caminho
        Se EncontrarCiclo(caminho, vizinho) == Verdadeiro:
          Retornar Verdadeiro
        Remover vizinho do caminho  // BACKTRACKING

    Retornar Falso  // Nenhum ciclo encontrado
```

Execução do Algoritmo (1/4): Início

Grafo G:



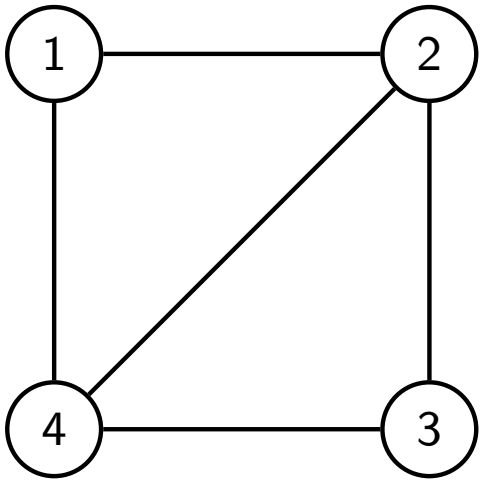
Pilha de Chamadas (Call Stack)

$$\text{CH}(1) \rightarrow [1]$$

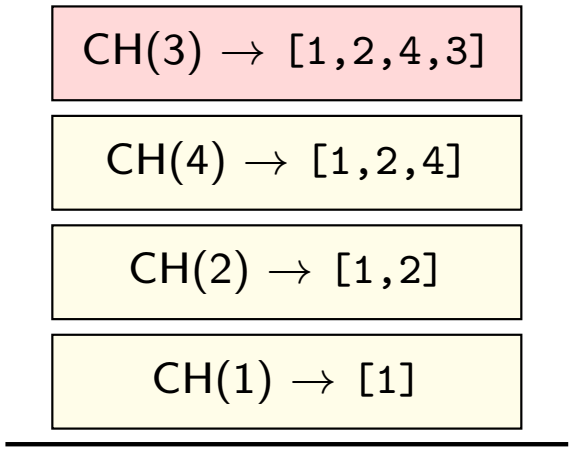
Fluxo: A execução começa chamando a função para o vértice 1. Ele é adicionado ao caminho e entra no topo da pilha na memória.

Execução do Algoritmo (2/4): Beco Sem Saída

Grafo G:



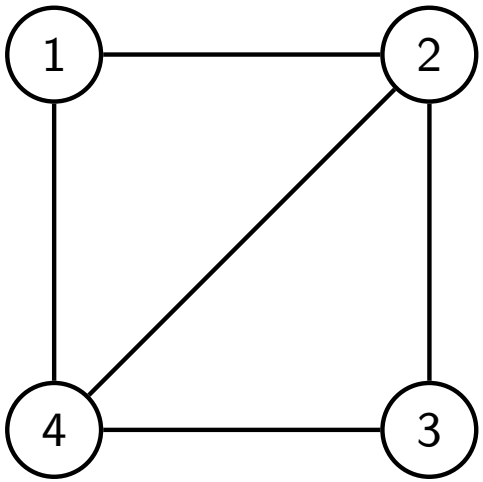
Pilha de Chamadas (Call Stack)



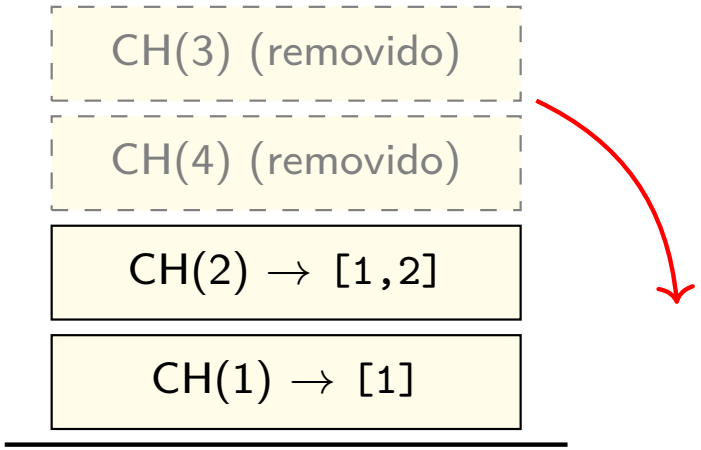
Fluxo: O algoritmo tenta o primeiro caminho disponível ($1 \to 2 \to 4 \to 3$). O limite de 4 vértices é atingido, mas **não existe** aresta ligando 3 de volta ao 1! A recursão retorna `FALSO`.

Execução do Algoritmo (3/4): Backtracking

Grafo G:



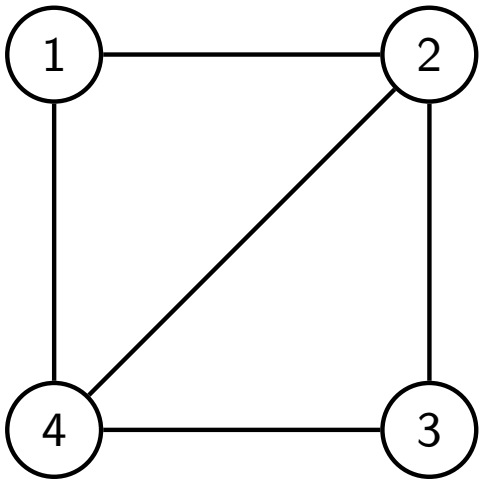
Pilha de Chamadas (Call Stack)



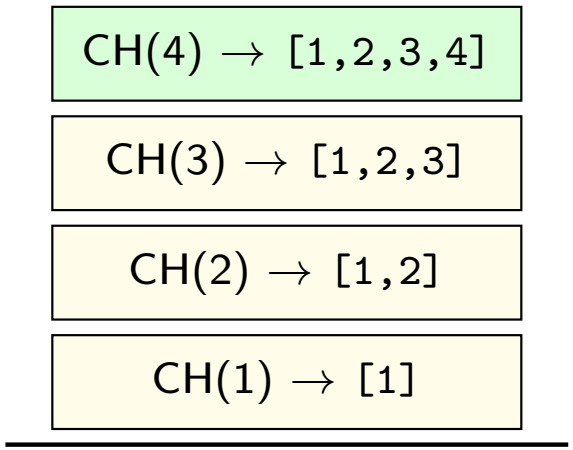
Fluxo: Como a rota falhou, a função `removerUltimo()` é ativada. Os nós 3 e 4 saem do caminho e são **desempilhados**. A execução recua ("backtracks") para o nó 2, pronto para tentar um outro vizinho.

Execução do Algoritmo (4/4): Sucesso

Grafo G:



Pilha de Chamadas (Call Stack)



Fluxo: A partir do 2, tenta-se o próximo vizinho: o 3, que tenta o 4. A pilha atinge o limite com a rota $1 \to 2 \to 3 \to 4$. Desta vez, há aresta conectando 4 ao 1! A função retorna `VERDADEIRO` em cascata e fecha o ciclo.